

A Grasshopper Optimization Algorithm Approach for Solving the Uncapacitated Facility Location Problem

1. Introduction

The Uncapacitated Facility Location Problem (UFLP) is a classic optimization challenge where we must decide:

- Which facilities to open (e.g., warehouses, schools, hospitals).
- How to assign customers to these facilities.

The goal is to minimize total costs, which include:

- Fixed costs (opening facilities).
- Transportation costs (serving customers from the nearest open facility).

This report explains how the Grasshopper Optimization Algorithm (GOA), a nature-inspired metaheuristic, can efficiently solve UFLP.

2. Problem Intuition & Real-World Analogy

A. UFLP as a City Planning Problem

Imagine you are a city planner tasked with:

- Building schools (facilities) in a city.
- Assigning students (customers) to the nearest school.

Your constraints:

- Each school has an opening cost (fixed cost).
- Students must travel to their nearest school (transportation cost).

Your goal:

Minimize the total cost = (Cost of building schools) + (Total student travel distance).

B. Why is UFLP Hard?

- NP-Hard: No efficient exact solution for large instances.
 - Combinatorial explosion: For N facilities, there are 2^N possible solutions.
-

A. Grasshopper Optimization Algorithm (GOA)

A. Biological Inspiration GOA mimics the swarming behavior of grasshoppers in nature:

- Baby grasshoppers (Exploration): Move randomly in large jumps to explore.
- Adult grasshoppers (Exploitation): Move slowly toward food (optimal solutions).

B. Mathematical Model Each grasshopper's position X_i updates using:

$$X_i = c \left(\sum_{j \neq i} s(d_{ij}) \cdot \frac{X_j - X_i}{d_{ij}} \right) + T_d$$

Where:

- c : Controls exploration vs. exploitation (starts high, decays over time).
- $s(r)$: Social force (attraction/repulsion between grasshoppers).
- T_d : Best solution found so far (target attraction).

In the implementation, GOA is adapted to a binary optimization setting. The continuous position updates generated by the algorithm are converted into binary facility-opening decisions using a sigmoid transfer function and stochastic thresholding.

C. Key Parameters

Parameter	Role	Typical Value
c_{max}	Initial exploration rate	1.0
c_{min}	Final exploitation rate	0.001
f	Social interaction intensity	0.3
l	Social interaction scale	1.5

B. GOA for UFLP: Step-by-Step

A. Solution Encoding

- Represent solutions as binary vectors: $[1, 0, 1] \rightarrow$ Open facilities 1 and 3, close facility 2.
- Since UFLP is a binary optimization problem, the continuous positions generated by GOA are converted into binary decisions using a sigmoid transfer function. A value of 1 indicates that a facility is opened, while 0 indicates that it remains closed.

B. Fitness Function

For a given solution Y:

$$Total\ Cost = \sum_{i \in opened} f_i + \sum_{j \in customers} \min_{i \in opened} (d_{ij})$$

C. GOA Workflow

- a. Initialize: Randomly generate binary solutions (grasshoppers).
- b. Evaluate: Compute costs for each solution.
- c. Update Positions:
 - Adjust solutions using social forces ($s(r)$) and target attraction (T_d).
 - Convert continuous updates to binary (sigmoid thresholding).
- d. Repeat: Until convergence or max iterations.

D. Binary Position Update

After computing the social interaction term, each grasshopper position is updated in a continuous space and then transformed into a binary vector using a sigmoid transfer function followed by stochastic sampling.

C. Experimental Results

The proposed Binary GOA approach was tested on a randomly generated UFLP instance containing 50 candidate facilities and 500 customers. The algorithm was executed for 100 iterations with a population size of 30 grasshoppers. Each facility opening cost and customer location was generated randomly using a fixed random seed.

The obtained results are summarized below:

Metric	Value
Best Cost	88.45
Runtime	1.34 sec
Opened Facilities	33
Greedy Cost	88.64
Greedy Opened Facilities	35
Improvement	0.21%

Mean Cost (10 runs)	88.51
Standard Deviation	0.27
Best Run	88.31
Worst Run	89.04

The results indicate that the proposed Binary GOA achieved a slightly lower total cost than the greedy baseline while opening fewer facilities. Furthermore, the small standard deviation across multiple runs suggests that the algorithm provides stable and consistent performance.

D. Discussion

A. Why GOA Works for UFLP

- Balanced Search: Adaptive c prevents premature convergence.
- Social Forces: Avoids overcrowding (repulsion) while encouraging good solutions (attraction).
- Scalability: Handles large instances where exact methods fail.

B. Limitations

- Parameter Sensitivity: Requires tuning c_{max} , f , etc.
- Binary Conversion: Sigmoid thresholding may flip bits aggressively.

E. Conclusion

GOA is a powerful metaheuristic for solving UFLP, offering:

- Near-optimal solutions at low computational cost.
- Interpretability through biological analogies.
- Flexibility for extension to more realistic facility location variants such as capacitated facility location problems and multi-objective facility planning scenarios.